

P2812: P1673R11 LEWG presentation

Document #: P2812
Date: 2023/02/10
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Nevin Liber
<nliber@anl.gov>

Contents

- 1 P1673R11 LEWG presentation
 - 1.1 Context
 - 1.2 Motivation
 - 1.2.1 Previous WG21 talks and proposals
 - 1.2.2 Very brief summary
 - 1.2.3 Example: matrix-vector product
 - 1.3 Previous reviews
 - 1.4 Implemented Feedback from Kona
 - 1.4.1 Specification of requirements on mdspan parameters
 - 1.4.2 Describe the algorithms mathematically
 - 1.4.3 Linear algebra value types
 - 1.5 Notes

§ 1 P1673R11 LEWG presentation

§ 1.1 Context

This paper is nonnormative. It is the presentation that P1673R11 coauthors gave to LEWG at the Issaquah WG21 meeting on 2023/02/10.

§ 1.2 Motivation

§ 1.2.1 Previous WG21 talks and proposals

- Today will focus on R11 changes and wording
- For more background, please see
 - Recorded talk (less than 20 mins): <https://youtu.be/n7mBGDqSzIQ>
 - Talks at Kona 2019 and elsewhere
 - Design justification in P1674 and P1673 intro
 - Historical background in P1417

§ 1.2.2 Very brief summary

- *Algorithms* operating on *views* of users' data
 - Verbs (named algorithms), not nouns (“matrix,” “vector”)
 - Views, not containers
 - Uses `mdspan` (C++23) as multidimensional array view ...
 - ... so that users can call algorithms with their existing data structures
- Extends BLAS (Basic Linear Algebra Subroutines)
 - Releases 1979, 1987, 1989; Standard 2002
 - Over four decades of continuous use
 - P1673's extensions
 - ANY data types, including mixed
 - ANY data layouts
 - Parallel algorithms (`ExecutionPolicy&&`)

§ 1.2.3 Example: matrix-vector product

```
using vector_t = mdspan<double, dextents<int, 1>>;
using matrix_t = mdspan<double, dextents<int, 2>>;

// ... allocate and fill ...
vector_t x = /* ... */;
vector_t y = /* ... */;
matrix_t A = /* ... */;

// compute y = Ax
matrix_vector_multiply(y, A, x);

// compute norm2 of y
double val = vector_norm2(y);

// mixed precision, different layout
```

```
mdspan<float, dextents<int, 2>, layout_left> A_f =
    /* allocate and fill */;
matrix_vector_multiply(y, A_f, x);
double val2 = vector_norm2(y);
```

§ 1.3 Previous reviews

- Seen multiple times by LEWG
- LEWG generally agreed on scope and functionality
- In Kona we mostly worked on figuring out how to word this properly

§ 1.4 Implemented Feedback from Kona

- Use exposition-only concepts to express constraints, rather than special template parameter names
- Describe effects of math algorithms in math notation

§ 1.4.1 Specification of requirements on mdspan parameters

Implements LEWG’s Kona 2022/11/10 request to

explore expressing constraints with concepts instead of named type requirements.

§ 1.4.1.1 Before

```
template<class in_vector_t,
         class in_matrix_t,
         class out_vector_t>
void matrix_vector_product(in_matrix_t A,
                          in_vector_t x,
                          out_vector_t y);
```

- C++98-style named-based requirements on template parameters
 - Imitating [algorithms.requirements] 4: “Throughout this Clause, where the template parameters are not constrained, the names of template parameters are used to express type requirements.”
 - Defined in [linalg.algs.reqs] in P1673R10
- “in_vector*_t is a rank-1 mdspan with a potentially const element type and a unique layout. If the algorithm accesses the object, it will do so in read-only fashion.”
- “out_vector*_t is a rank-1 mdspan with a non-const element type, and whose layout is always unique (layout_type::is_always_unique() equals true). If the algorithm accesses the object, it will do so in write-only fashion.”
- For multiple parameters: in_matrix_1_t, in_matrix_2_t, etc.

§ 1.4.1.2 After

```
template<in-matrix InMat,  
        in-vector InVec,  
        out-vector OutVec>  
void matrix_vector_product(InMat A,  
                          InVec x,  
                          OutVec y);
```

- C++ has concepts now; let's use them
- Exposition-only concepts
 - Defined in [linalg.concepts]
 - Only observable as constraints
 - No new concepts exposed to users
- Also: Remove uniqueness requirement on input objects

§ 1.4.1.3 Exposition-only concept example

```
template<class T>  
struct is-mdspan : false_type {}; // exposition only  
  
template<class ElementType, class Extents, class Layout, class Accessor>  
struct is-mdspan<mdspan<ElementType, Extents, Layout, Accessor>> //  
    exposition only  
    : true_type {};  
  
template<class T>  
concept in-vector = // exposition only  
    is-mdspan<T>::value &&  
    T::rank() == 1;  
  
template<class T>  
concept out-vector = // exposition only  
    is-mdspan<T>::value &&  
    T::rank() == 1 &&  
    is_same_v<remove_const_t<typename T::element_type>, typename  
        T::element_type> &&  
    T::is_always_unique();  
  
// ...  
  
template<class T>  
concept in-matrix = // exposition only  
    is-mdspan<T>::value &&  
    T::rank() == 2;
```

Entirely syntactic, except for:

- 2 If an algorithm in [linalg.algs] accesses the elements of an *in-vector*, *in-matrix*, or *in-object*, it will do so in read-only fashion.

§ 1.4.2 Describe the algorithms mathematically

Follows LEWG guidance to simplify Effects and Constraints.

- Before: “the mathematical expression for the algorithm”
 - a code-font expression that implies constraints
- After: describe algorithms mathematically
 - Mathematical expression, in math font

This is our interpretation of the “hand wavy do math” poll option that received a majority of votes at Kona on 2022/11/10.

§ 1.4.2.1 P1673 takes any “number-like” types

- P1673 takes any “number-like” types
 - NOT just the BLAS’s 4 types
 - NOT just floating-point types
 - NOT just arithmetic and complex types
- This includes
 - Short floats (like `float16_t`), bigfloats
 - Booleans, integers, fractions
 - Types with no division, like polynomials
 - Types with noncommutative multiplication, like quaternions
 - Mixed precision (BLAS Standard Chapter 4)
 - Any combinations of these that make mathematical sense
- All of these are useful in practice

§ 1.4.2.2 This imposes a wording challenge

- P1673 (or anything like it) generalizes `std::reduce`
 - Can evaluate sums in any order, like `GENERALIZED_SUM` [numerics.defns]
 - Can read and write `mdspan` elements multiple times
 - Can create temporaries
- Constraining value types with concepts is not a good fit
 - Several LEWG reviews have accepted this
 - “Can reorder sums” != “plus is associative”
 - Not just rounding error; e.g., saturating arithmetic, where error can be unbounded

— Yes, “concepts constrain algorithms,” but concepts are Boolean; they can’t lie

— Expressions may mix types or involve intermediate expression templates

Before	After
For i in the domain of y and N equal to $A.\text{extent}(1)$, the mathematical expression for the algorithm is $y[i] = \text{the sum of } A[i, j] * x[j] \text{ for all } j \text{ such that } i, j \text{ is in the domain of } A.$	<i>Effects:</i> Computes y such that $y = Ax$.

P1673R10 defined “Mathematical expression for the algorithm” as:

Each algorithm or method using the type has one or more associated mathematical expressions that defines the algorithm’s or method’s behavior. For each algorithm or method, its mathematical expression(s) are either explicitly stated as such, or are implicitly stated in the algorithm’s or method’s description. The requirements below will refer to those mathematical expression(s).

P1673R11’s Effects reference the mathematical definition of matrix-vector multiplication. Compare to wording for the mathematical special functions [sf.cmath], which uses math-font integrals, derivatives, and infinite sums. [linalg.reqs.val] is much shorter.

§ 1.4.3 Linear algebra value types

- Algorithms are generic on value types
- [linalg.reqs.val] explains requirements on “linear algebra value types”

§ 1.5 Notes

- Need to add feature test macro: `__cpp_lib_linalg`